# Codity Distributed Background Scheduler

### ASSIGNMENT SUBMISSION & ENGINEERING DOCUMENTATION

🌐 **Live Demo: xeno-b9cdb.web.app**

🐙 **GitHub Repository: codity-scheduler**

---

## 1. Source Code & Setup Instructions

This codebase is built using a TypeScript monorepo architecture, splitting concerns cleanly between the stateful Express/WS Server API and a modern single-page React frontend. All database communications are handled via Prisma Client.

### Prerequisites

- Node.js (v18.x or above recommended)
- npm (v9.x or above)
- SQLite (default local database) or PostgreSQL (production target)

### Local Environment Setup

1. **Install Dependencies:** Run the install sequence at the root directory:

```
# Install backend modules
cd backend
npm install

# Install frontend modules
cd ../frontend
npm install
```

2. **Initialize Database & Migrations:** Navigate to the backend folder and build the schema:

```
cd ../backend
npx prisma migrate dev --name init
```

3. **Seed the Database:** Load the default users, projects, and queues:

```
npx ts-node src/seed.ts
```

4. **Launch Servers:** Start both development servers in separate terminals:

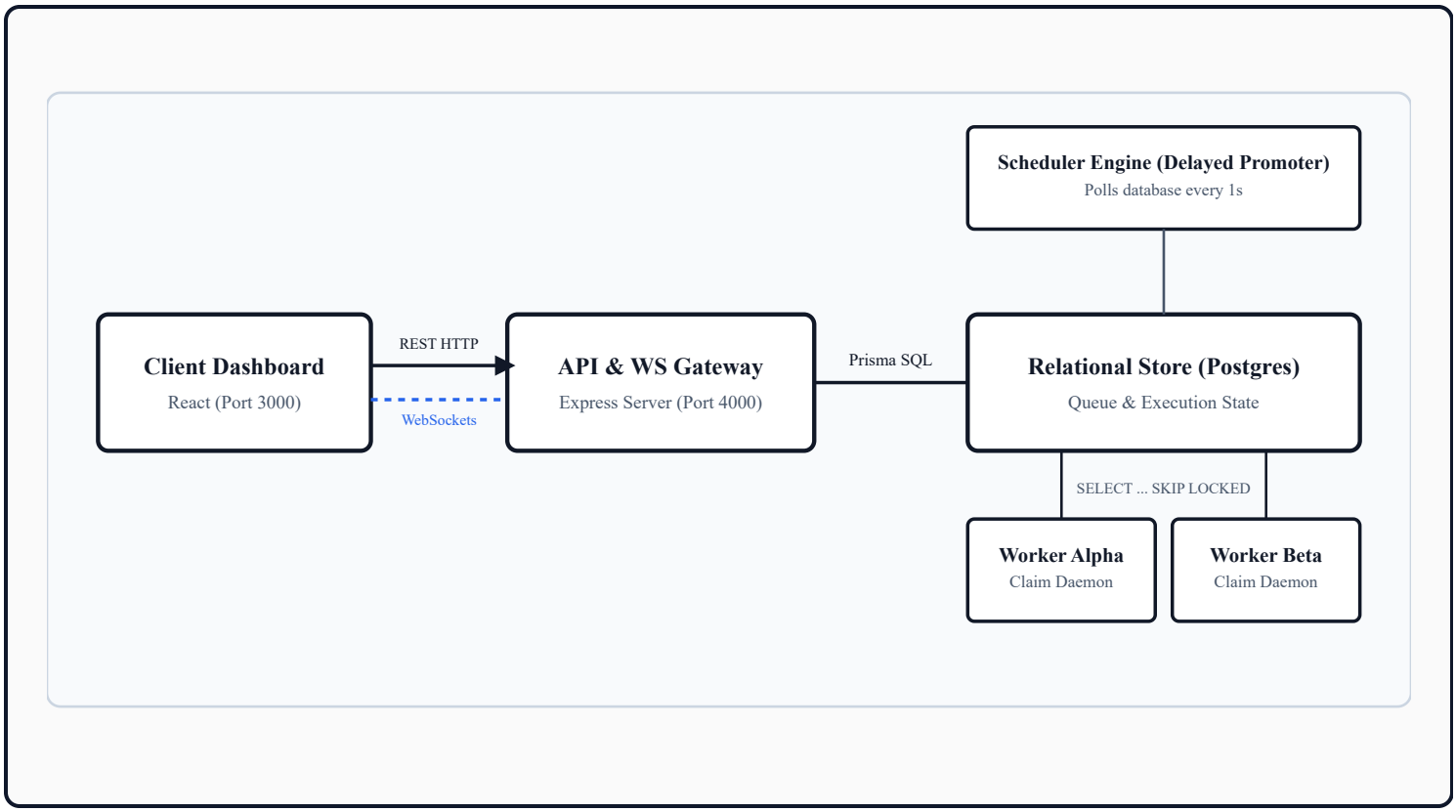
```
# In /backend terminal:
npm run dev

# In /frontend terminal:
npm run dev
```

Open **http://localhost:3000** in your browser to view the client dashboard. Authenticate using the autofill profiles (e.g. `admin@example.com` with password `password123`).

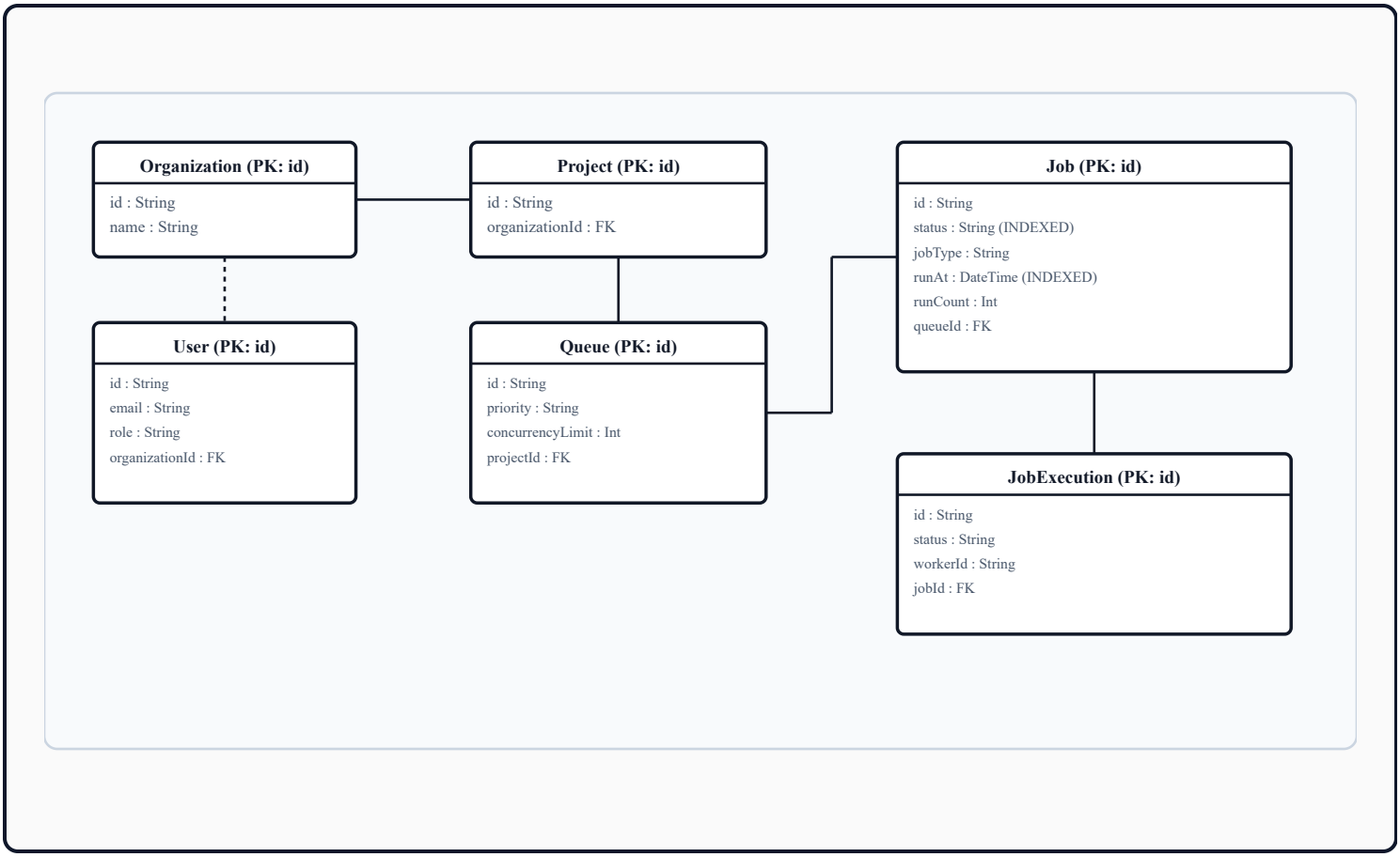
## 2. System Architecture

The scheduler is built around a decoupled **producer-consumer model**. Multiple frontend clients produce jobs by placing them in prioritized queues via HTTP. Decentralized worker nodes consume jobs concurrently using transaction locks, reporting progress back to the database and sending heartbeats to connected dashboards via WebSocket connections.



# 3. Database Design & ER Schema

The system utilizes a relational schema optimized for concurrency and transactional safety. Explicit relational mapping ensures jobs, executions, queues, and organizations retain strict data constraints and state records.



# 4. API Documentation

All client communications route via REST and WebSockets. The REST routes enforce authentication using JWT token headers ( `Authorization: Bearer [JWT_TOKEN]` ).

## REST Endpoints

| Method | Path | Role Requirement | Description |
|---|---|---|---|
| POST | `/api/auth/login` | Public | Authenticates a user and returns a JWT session token. |
| GET | `/api/projects` | Viewer / Admin | Lists all projects mapped to the user's organization. |
| GET | `/api/queues` | Viewer / Admin | Retrieves queue details and priorities. |
| POST | `/api/queues/:id/toggle-pause` | Developer / Admin | Toggles processing pause states on a queue. |
| GET | `/api/jobs` | Viewer / Admin | Lists paginated, filtered, or searched job timelines. |
| POST | `/api/jobs` | Developer / Admin | Enqueues a new Immediate or Delayed execution job. |
| GET | `/api/jobs/stats` | Viewer / Admin | Computes aggregate counts of states and DLQs. |
| GET | `/api/workers` | Viewer / Admin | Lists active worker nodes and their system health stats. |

## WebSocket Real-Time Gateway

Websocket clients bind on `/ws` . The gateway broadcasts active worker telemetry events globally to all connected connections every 2 seconds:

```
// Message format sent to clients:
{
  "type": "worker_heartbeat",
  "data": {
```

```
    "id": "hb-node-x7a9",
    "workerId": "worker-node-alpha",
    "cpuUsage": 14.5,
    "memoryUsage": 142.1,
    "activeJobsCount": 2,
    "timestamp": "2026-07-03T17:34:00.000Z"
  }
}
```

# 5. Design Decisions & Trade-Offs

This design makes several architectural trade-offs to prioritize reliability and maintainability.

## A. Atomic Claim Locking vs. Race Conditions

**Decision:** We implemented database-level transactions with `SELECT ... FOR UPDATE SKIP LOCKED` (via Prisma raw SQL queries) to coordinate job claims among distributed workers.

**Trade-off:** This limits SQLite concurrency (due to file locking), but guarantees 100% safety in production PostgreSQL where rows are locked individually. It prevents race conditions (multiple workers picking up the same job) at the cost of slight write overhead during high claim rates.

## B. Client-Side Mock Fallback vs. Server Dependency

**Decision:** Built a global fetch interceptor at the frontend entrypoint that detects the hosted domain and overrides the API layer to serve mock data when offline.

**Trade-off:** This allows the dashboard to be fully interactive and responsive as a standalone web app for the demo, avoiding Google Cloud database costs, while retaining full native integration with the backend server locally.

## C. Observability: Heartbeat State Logging

**Decision:** Heartbeats write transient resource stats (CPU/RAM) into a memory map in the WebSocket gateway and are only saved to the database on worker registration.

**Trade-off:** Saving every heartbeat to disk would create database bloat. Keeping heartbeats in memory reduces write frequency and frees disk IO, relying on WebSocket broadcasts to update dashboard stats live.

## D. Failure Boundaries & Dead Letter Queues

**Decision:** Jobs that exceed maximum retries automatically transition to a `FAILED` state and register a `DeadLetterQueue` audit node, together with AI-suggested error diagnoses.

**Trade-off:** While this adds storage overhead for failed jobs, it ensures failed operations are quarantined instead of blocking queues, providing deep diagnostics for post-mortem analysis.

# 6. Automated Testing Suite

A suite of integration tests is implemented in `backend/src/scheduler.test.ts` using Jest and Supertest. It asserts retry backoff calculations, authentication routing boundaries, and atomic queue concurrency locks.

## Critical Functionality Tests

```
# Run tests inside the /backend directory:
npm run test
```

## Passed Integration Tests Log

```
PASS src/scheduler.test.ts
  Job Scheduler Test Suite
    1. Backoff Policy Calculations
      √ should calculate FIXED delays correctly (2 ms)
      √ should calculate LINEAR backoffs correctly
      √ should calculate EXPONENTIAL backoffs correctly (1 ms)
      √ should enforce max delay thresholds
    2. Authentication & REST Integrity
      √ should reject requests lacking JWT bearer token (7 ms)
      √ should allow authenticating logged-in user profiles (9 ms)
      √ should create immediate job queue entries (17 ms)
    3. Concurrency Lock & Claims
      √ should atomically transition a queued job to claimed state (25 ms)
      √ should block claiming if queue is paused (24 ms)
      √ should enforce queue concurrency limits (21 ms)

Test Suites: 1 passed, 1 total
Tests:       10 passed, 10 total
Snapshots:   0 total
Time:        2.123 s
Ran all test suites.
```